iVolve DevOps Training — Complete Lab Guide

Labs 1–3 · Build Tools and Containerization Eslam Nabawy · Ubuntu 24.04 on WSL2 · October 2026

Table of contents

1. Foundations — what you need to understand first
2. The application under test
3. Lab 1 — Maven
4. Lab 2 — Gradle
5. Lab 3 — Docker
6. Comparison and conclusions
7. Troubleshooting
8. Glossary

1. Foundations

Before the labs, five concepts. Everything that follows is built on them.

1.1 JDK, JRE and JVM

These three get used interchangeably and shouldn't be. The distinction is the whole point of Lab 3.

Term	What it is	Contains	Size
JVM	Java Virtual Machine — executes bytecode	the execution engine	—
JRE	Java Runtime Environment — what you need to **run**	JVM + standard libraries	~80–180 MB
JDK	Java Development Kit — what you need to **build**	JRE + compiler (`javac`) + tools	~180–400 MB

JDK ⊃ JRE ⊃ JVM. You compile with a JDK; you run with a JRE. A production server that only runs an application does not need a compiler — that is exactly the insight Lab 3 Part 2 makes you discover by measuring it.

1.2 Source code vs artifact

Java source (`.java`) is not executable. The chain:

```
Calculator.java  --javac-->  Calculator.class  --package-->  calculator.jar
   (source)                    (bytecode)                      (artifact)
```

A `.jar` (Java ARchive) is a ZIP file containing `.class` bytecode plus a manifest. Prove it:

```
unzip -l target/calculator.jar
```

The **artifact** is the single distributable file produced by a build. Deployment moves artifacts, not source.

1.3 What a build tool actually does

A build tool automates source → artifact, **reproducibly**. Doing it by hand means running `javac` with the right classpath, downloading dependency JARs yourself, running tests, and zipping with a correct manifest. A build tool does all of it from one declarative file.

Three jobs:

1. **Dependency management** — fetch libraries (and *their* dependencies, transitively) from a repository such as Maven Central.
2. **Lifecycle execution** — compile → test → package, in order, with failure stopping the line.
3. **Reproducibility** — same source + same config = same artifact, on any machine.

1.4 Container vs image vs Dockerfile

Term	What it is	Analogy
Dockerfile	text recipe for building an image	the recipe
Image	immutable, layered filesystem snapshot	the cake
Container	a running instance of an image	the slice being eaten

One image spawns many containers. Containers are disposable; the image is the artifact you store and ship.

Layers matter. Each Dockerfile instruction creates a layer. Layers are cached and shared between images — two images on the same base store that base **once**. Instruction order therefore changes both build speed and image size.

1.5 Container vs virtual machine

A VM virtualizes **hardware** and runs a full guest OS — gigabytes, boots in minutes. A container virtualizes the **OS** — it shares the host kernel and isolates only the process tree, filesystem and network, using Linux namespaces and cgroups. Megabytes, starts in milliseconds.

This is why a container cannot run a different kernel than its host, and why Docker on Windows runs a small Linux VM behind the scenes — which in your case is WSL2 itself.

---

2. The application under test

All three labs build the same program, deliberately: the app is held constant so the *tooling* is the variable.

2.1 Source

```
// Calculator.java — the logic
public class Calculator {
    public double sum(double a, double b)      { return a + b; }
    public double subtract(double a, double b) { return a - b; }
    public double multiply(double a, double b) { return a * b; }
    public double divide(double a, double b) {
        if (b == 0) throw new ArithmeticException("Cannot divide by zero");
        return a / b;
    }
}
```

```
// Main.java — the entry point
Scanner scanner = new Scanner(System.in);
System.out.print("Enter first number: ");
double a = scanner.nextDouble();
System.out.print("Enter second number: ");
double b = scanner.nextDouble();
// ... prints sum, subtract, multiply, divide
```

2.2 The single most important fact about this app

It is an interactive command-line program that reads standard input. It is not a web server. Nothing listens on any port.

Three consequences you will meet directly:

1. `EXPOSE 8080` **in Lab 3 does nothing.** `EXPOSE` is metadata — a label recorded in the image saying "this is the port I would use". It opens no socket and starts no listener. The lab asks for it as a habit-forming step; include it, but know what it is.
2. **Verification is not** `curl localhost:8080`**.** It is: run it, type two numbers, confirm the arithmetic.
3. **The container needs** `docker run -i`**.** Without `--interactive` the container's stdin is closed, `scanner.nextDouble()` immediately hits end-of-file, and the app dies with `NoSuchElementException`. This *will* happen to you if you forget the flag.

2.3 Separation of logic and I/O

`Calculator` does arithmetic and nothing else; `Main` does all the input/output. That separation is why `Calculator` is unit-testable — tests call `sum(2,3)` directly with no keyboard involved. Code that mixed arithmetic into `Main` could not be tested this way. Worth noticing, because it is the reason the test suite exists at all.

---

3. Lab 1 — Building and packaging with Maven

Source: `https://github.com/Ibrahim-Adel15/CalculatorMavenApp.git` **Goal:** install Java and Maven, clone, test, build `target/calculator.jar`, run, verify.

3.1 What Maven is

Apache Maven is a declarative, convention-over-configuration build tool. You describe what the project **is** in `pom.xml` (Project Object Model); Maven applies a fixed lifecycle. You do not write build steps — you declare facts and Maven infers the steps.

The conventions it assumes by default:

Path	Meaning
`src/main/java`	application source
`src/test/java`	test source
`target/`	all build output
`pom.xml`	the project descriptor

Follow the convention and the config stays tiny. Fight it and you suffer. That trade is Maven's entire design philosophy.

3.2 Step 1 — Install Java 17 and Maven

```
sudo apt update && sudo apt install -y openjdk-17-jdk maven
```

- `apt update` refreshes the package index (what versions exist), it does not upgrade anything.
- `openjdk-17-jdk` is the **full JDK** — needed because we compile. `openjdk-17-jre` would not be enough.
- `-y` auto-confirms prompts.

Verify:

```
java -version && mvn -version
```

The critical line is in the `mvn -version` output:

```
Java version: 17.0.20.1, vendor: Ubuntu, runtime: /usr/lib/jvm/java-17-openjdk-amd64
```

It must say **17**. `pom.xml` sets `maven.compiler.source` to 17; an older JDK fails the build. If it reports the wrong version, Maven is finding a different JDK — pin it:

```
echo 'export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64' >> ~/.bashrc && source ~/.bashrc
```

`JAVA_HOME` is the environment variable every JVM tool reads to locate the JDK.

3.3 Step 2 — Clone the source

```
mkdir -p ~/labs-workspace && git clone https://github.com/Ibrahim-Adel15/CalculatorMavenApp.git ~/labs-workspace/CalculatorMavenApp
```

Why outside the repo: your submission repo should contain *your work* — Dockerfiles, configs, write-ups. Not a copy of someone else's source, and not build output. Keeping clones in `~/labs-workspace/` makes accidental commits impossible.

Inspect before building:

```
find ~/labs-workspace/CalculatorMavenApp -type f -not -path '*/.git/*'
```

Four files: `pom.xml`, `Main.java`, `Calculator.java`, `CalculatorTest.java`. A whole build system driven by one 60-line XML file.

3.4 Step 3 — Understand `pom.xml` before running it

```
<groupId>com.example</groupId>        <!-- org namespace, reverse-domain -->
<artifactId>calculator</artifactId>   <!-- project name -->
<version>1.0.0</version>              <!-- version -->
<packaging>jar</packaging>            <!-- output type -->
```

Those four are the **coordinates** — the globally unique address of an artifact. `com.example:calculator:1.0.0`. Every dependency you declare is addressed the same way.

```
<properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
</properties>
```

`source` = which language syntax is accepted. `target` = which bytecode version is emitted. Setting both to 17 keeps them consistent.

```
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.2</version>
    <scope>test</scope>
</dependency>
```

`<scope>test</scope>` is the line worth studying. It puts JUnit on the classpath for compiling and running tests, and *excludes it from the runtime classpath*. A test framework in a production artifact is pure waste and extra attack surface. Maven enforces the separation declaratively.

```
<build>
    <finalName>calculator</finalName>
```

Why the jar is `calculator.jar`. Maven's default output name is `artifactId-version.jar` → `calculator-1.0.0.jar`. The lab demands exactly `calculator.jar`; `<finalName>` overrides the default.

```
<plugin>
    <artifactId>maven-jar-plugin</artifactId>
    <configuration><archive><manifest>
        <mainClass>com.example.calculator.Main</mainClass>
    </manifest></archive></configuration>
</plugin>
```

Why `java -jar` works. A JAR is a ZIP; the JVM only knows where to start because `META-INF/MANIFEST.MF` contains a `Main-Class` line. This plugin writes it. Delete this block and you get `no main manifest attribute`.

3.5 Step 4 — Run the unit tests

```
cd ~/labs-workspace/CalculatorMavenApp && mvn test
```

The first run downloads JUnit and all plugins into `~/.m2/repository` — slow once, cached after. Success looks like:

```
Tests run: N, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Failures vs Errors — a *failure* is an assertion that did not hold (the code is wrong). An *error* is an unexpected exception (the test itself blew up). Different causes, different fixes.

Read the test file — it covers the error path, not just the happy path:

```
cat ~/labs-workspace/CalculatorMavenApp/src/test/java/com/example/calculator/CalculatorTest.java
```

There is a test asserting that `divide(x, 0)` throws `ArithmeticException`. Testing that something *fails correctly* is as important as testing success.

3.6 Step 5 — Build the artifact

```
cd ~/labs-workspace/CalculatorMavenApp && mvn clean package
```

Two goals, run in order:

- `clean` deletes `target/`. Without it you risk shipping a stale class from a previous build — a genuinely nasty class of bug.
- `package` runs the lifecycle up to and including packaging.

The Maven lifecycle

Phases run **cumulatively** — naming one runs every phase before it:

Phase	Does
`validate`	check the project is correct
`compile`	`.java` → `.class` into `target/classes`
`test`	run unit tests
`package`	assemble into `target/calculator.jar`
`verify`	run integration checks
`install`	copy artifact to local `~/.m2`
`deploy`	upload to a remote repository

So `mvn package` **also compiles and tests**. If a test fails, packaging never happens — the pipeline refuses to produce a broken artifact. That property is the foundation of CI.

Confirm the artifact:

```
ls -lh ~/labs-workspace/CalculatorMavenApp/target/calculator.jar
```

~3.7 KB — tiny, because it contains only your bytecode. The JVM and standard library live outside it.

3.7 Step 6 — Run and verify

```
java -jar ~/labs-workspace/CalculatorMavenApp/target/calculator.jar
```

Type `10`, Enter, `5`, Enter. Expected:

```
a = 10.0, b = 5.0
Sum: 15.0
Subtract: 5.0
Multiply: 50.0
Divide: 2.0
```

Values are `double`, which is why everything prints with `.0`.

Non-interactively — useful in CI, where no human types anything:

```
printf '10\n5\n' | java -jar ~/labs-workspace/CalculatorMavenApp/target/calculator.jar
```

`printf` writes to stdout; `|` pipes it into the program's stdin; `\n` is the Enter key. This is exactly what `docker run -i` provides and what its absence removes.

3.8 Step 7 — Prove the manifest claim

```
unzip -p ~/labs-workspace/CalculatorMavenApp/target/calculator.jar META-INF/MANIFEST.MF
```

```
Manifest-Version: 1.0
Created-By: Maven JAR Plugin 3.4.1
Build-Jdk-Spec: 17
Main-Class: com.example.calculator.Main
```

`unzip -p` extracts one file to stdout without unpacking the archive — direct evidence that a JAR is a ZIP and that the `<mainClass>` config landed where it was supposed to.

3.9 Lab 1 command summary

Command	Purpose
`mvn clean`	delete `target/`
`mvn compile`	source → bytecode
`mvn test`	compile + run tests
`mvn package`	compile + test + build jar
`mvn install`	package + copy to `~/.m2`
`mvn -version`	verify Maven and its JDK
`java -jar <file>`	run an executable jar

---

4. Lab 2 — Building and packaging with Gradle

Source: `https://github.com/Ibrahim-Adel15/CalculatorGradleApp.git` **Goal:** install Gradle, clone, test, build `build/libs/calculator.jar`, run, verify.

> **Correction to the lab slide.** The title reads "Building and Packaging **Go** Application with Gradle". The repository is **Java** — the same `Calculator.java` and `Main.java` as Lab 1. The slide has a copy-paste error. Build it as Java.

4.1 What Gradle is, and how it differs

Gradle is an imperative, programmable build tool. `build.gradle` is **executable code** (Groovy or Kotlin), not a static description. Running it constructs a **task graph** — a DAG where each task declares its inputs and outputs — and Gradle executes only the tasks whose inputs changed.

That one design decision produces every difference you will observe:

	Maven	Gradle
Config file	`pom.xml`	`build.gradle` + `settings.gradle`
Language	XML	Groovy / Kotlin DSL
Model	fixed lifecycle	task graph (DAG)
Output directory	`target/`	`build/`
Build command	`mvn package`	`gradle build`
Incremental builds	minimal	yes — skips up-to-date tasks
Background daemon	no	yes — JVM stays warm
Build cache	no	yes, local and remote
Verbosity	high	terse by default

Why two config files? `settings.gradle` declares *which projects* form the build (essential for multi-module projects); `build.gradle` declares *how to build* one project. Maven packs both roles into `pom.xml` using `<modules>`.

4.2 Step 1 — Install Gradle (not with apt)

Do not run `sudo apt install gradle`. Ubuntu 24.04 ships Gradle 4.4.1 — from 2017. This project's `build.gradle` uses the `toolchain` block, introduced in Gradle 6.7. The apt version fails with a confusing error about an unknown method.

Use **SDKMAN**, the standard JVM-ecosystem version manager:

```
curl -s "https://get.sdkman.io" | bash && source "$HOME/.sdkman/bin/sdkman-init.sh"
```

```
sdk install gradle
```

```
gradle -version
```

- `curl -s` fetches the installer quietly; piping to `bash` runs it. (Piping a URL into a shell deserves caution in general — read the script first if you do not trust the source. SDKMAN is a well-known, widely used project.)
- `source` re-reads the init script into your **current** shell, so `sdk` works without reopening the terminal.
- SDKMAN installs into `~/.sdkman` — entirely in your home directory, no `sudo` required.

Why this matters beyond Gradle: SDKMAN also manages multiple JDKs side by side (`sdk list java`, `sdk use java 21-tem`). Real projects target different Java versions, and switching system-wide JDKs with `apt` is painful. This is how JVM developers actually work.

4.3 Step 2 — Clone and inspect

```
git clone https://github.com/Ibrahim-Adel15/CalculatorGradleApp.git ~/labs-workspace/CalculatorGradleApp
```

```
find ~/labs-workspace/CalculatorGradleApp -type f -not -path '*/.git/*'
```

Same `src/main/java` and `src/test/java` layout — Gradle adopted Maven's directory convention because it had already won.

4.4 Step 3 — Understand `build.gradle`

```
plugins {
    id 'java'
    id 'application'
}
```

Gradle ships almost no behaviour by default; **plugins add tasks**. The `java` plugin contributes `compileJava`, `test`, `jar`, `build`. The `application` plugin adds `run` and distribution packaging. Without `java`, `gradle build` would do nothing at all.

```
java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(17)
    }
}
```

The toolchain block has no Maven equivalent. It tells Gradle: compile against Java 17, and *if this machine has no JDK 17, download one automatically*. The build becomes independent of whatever JDK happens to be installed — a genuine reproducibility guarantee across a team and CI.

```
repositories { mavenCentral() }
```

Where to fetch dependencies. Gradle uses Maven Central too — the repository format is shared infrastructure, not Maven-specific.

```
dependencies {
    testImplementation platform('org.junit:junit-bom:5.10.2')
    testImplementation 'org.junit.jupiter:junit-jupiter'
    testRuntimeOnly "org.junit.platform:junit-platform-launcher"
}
```

- `testImplementation` is Gradle's equivalent of Maven's `<scope>test</scope>`.
- `platform(...)` imports a **BOM** (Bill of Materials) — a version catalogue. It is why the next line needs no version: the BOM supplies a consistent set, preventing mismatched JUnit modules.
- `testRuntimeOnly` means present when tests *run*, absent when they *compile*. Gradle's dependency scopes are finer-grained than Maven's.

```
test { useJUnitPlatform() }
```

The most dangerous line in the file if omitted. Gradle defaults to the JUnit 4 runner. Without this, JUnit 5 tests are not discovered — Gradle reports `BUILD SUCCESSFUL` having run **zero** tests. A green build that tested nothing is worse than a red one, because nobody investigates it.

```
jar {
    archiveBaseName = 'calculator'
    archiveVersion = ''
    manifest { attributes 'Main-Class': 'com.example.calculator.Main' }
}
```

Same two jobs as Maven's `<finalName>` and `maven-jar-plugin`: name the file `calculator.jar` (the empty `archiveVersion` strips the `-1.0.0` suffix) and write `Main-Class` into the manifest.

4.5 Step 4 — Run the tests

```
cd ~/labs-workspace/CalculatorGradleApp && gradle test
```

Gradle is deliberately terse — on success you get little more than `BUILD SUCCESSFUL`. For per-test detail:

```
cd ~/labs-workspace/CalculatorGradleApp && gradle test -i
```

An HTML report is always generated:

```
ls ~/labs-workspace/CalculatorGradleApp/build/reports/tests/test/index.html
```

Open it from WSL in your Windows browser:

```
explorer.exe "$(wslpath -w ~/labs-workspace/CalculatorGradleApp/build/reports/tests/test/index.html)"
```

`wslpath -w` converts a Linux path to its Windows form — one of the genuinely useful WSL interop tools.

4.6 Step 5 — Build

```
cd ~/labs-workspace/CalculatorGradleApp && gradle clean build
```

```
ls -lh ~/labs-workspace/CalculatorGradleApp/build/libs/calculator.jar
```

`build` is an aggregate task: `compileJava` → `processResources` → `classes` → `jar` → `test` → `check` → `build`.

Watch incremental builds work

Run it twice with no changes:

```
cd ~/labs-workspace/CalculatorGradleApp && gradle build && echo "=== SECOND RUN ===" && gradle build
```

The second run prints `UP-TO-DATE` beside most tasks and finishes in a fraction of the time. Gradle hashes each task's inputs; unchanged inputs mean the task is skipped entirely. Maven re-does the work every time. On a four-file project this saves seconds — on a large codebase it is the difference between a 40-second and a 12-minute build.

4.7 Step 6 — Run and verify

```
java -jar ~/labs-workspace/CalculatorGradleApp/build/libs/calculator.jar
```

Or via the `application` plugin:

```
cd ~/labs-workspace/CalculatorGradleApp && gradle run --console=plain -q
```

`--console=plain` is required for interactive programs — Gradle's rich console redraws progress bars over your prompt and makes input unreadable. `-q` suppresses everything but program output.

Inputs `10` and `5` produce identical results to Lab 1 — `Sum: 15.0`, `Subtract: 5.0`, `Multiply: 50.0`, `Divide: 2.0` — because it is the same source. **That is the lesson**: the build tool changes, the artifact's behaviour does not.

4.8 The wrapper — what this repo is missing

Production projects commit `gradlew`, `gradlew.bat` and `gradle/wrapper/`, then everyone runs `./gradlew build` instead of `gradle build`. The wrapper is a small script that downloads and uses an **exact pinned Gradle version** per project.

Why it matters: a new contributor needs no Gradle installed, and nobody can build with a different version and get different results. Maven has the same thing (`mvnw`). Neither of these lab repos includes one — a reasonable observation to note in your write-up, and a real-world practice worth adopting.

4.9 Lab 2 command summary

Command	Purpose
`gradle clean`	delete `build/`
`gradle test`	run tests
`gradle build`	compile + test + jar
`gradle run`	run via application plugin
`gradle tasks`	list all available tasks
`gradle build -i`	build with info-level logging
`sdk list java`	show installable JDKs

5. Lab 3 — Creating a Docker image

Source: `https://github.com/Ibrahim-Adel15/CalculatorMavenApp.git` **Goal:** build the same app two ways, compare image sizes, explain the difference.

5.1 Why this lab exists

The two parts are not busywork. They demonstrate a decision every containerized project faces: **where does the build happen?** Part 1 builds inside the image; Part 2 builds on the host. You measure both, and the size gap teaches the principle more convincingly than any explanation.

5.2 Dockerfile instructions you will use

Instruction	Purpose	Creates a layer?
`FROM`	base image to start from	yes
`WORKDIR`	set working directory, creating it if absent	metadata
`RUN`	execute a command at **build** time	yes
`COPY`	copy from build context into the image	yes
`USER`	set the user for subsequent instructions and runtime	metadata
`EXPOSE`	document a port	metadata only
`ENTRYPOINT`	the command run at **container start**	metadata
`CMD`	default arguments, or default command	metadata

`RUN` vs `ENTRYPOINT` is the distinction people get wrong. `RUN` executes while the image is being *built*, and its result is baked into a layer. `ENTRYPOINT` executes when a container *starts* — it is not run during the build at all.

`ENTRYPOINT` vs `CMD`: `ENTRYPOINT` is the fixed command; `CMD` supplies default arguments that `docker run` can override. `ENTRYPOINT ["java","-jar","/app/calculator.jar"]` means the container always runs the app.

Exec form vs shell form. Always write `ENTRYPOINT ["java", "-jar", "app.jar"]` (JSON array, exec form), not `ENTRYPOINT java -jar app.jar` (shell form). Shell form wraps the process in `/bin/sh -c`, so your app becomes PID 2 and **never receives signals** — `docker stop` then hangs for 10 seconds and kills it ungracefully. Exec form makes your process PID 1 and signals reach it properly.

5.3 The build context

```
docker build -f Dockerfile.app1 -t app1 .
```

That trailing `.` is the **build context** — the directory sent to the Docker daemon before the build starts. `COPY` can only read from inside it. A large context means a slow build even if you copy nothing, because the whole directory is transferred first.

`.dockerignore` excludes files from the context, exactly like `.gitignore`:

```
printf 'src/\n.git/\ntarget/*\n!target/calculator.jar\nDockerfile*\n' > ~/labs-
workspace/CalculatorMavenApp/.dockerignore
```

The `!target/calculator.jar` line re-includes one file after excluding its directory — order matters, later rules win.

5.4 Preparation

```
docker ps
```

If this errors with "cannot connect to the Docker daemon":

```
sudo service docker start
```

WSL has no systemd by default, so the daemon does not reliably auto-start at boot.

If it says "permission denied", add yourself to the `docker` group, then **close and reopen the terminal** (group membership is read at login):

```
sudo usermod -aG docker $USER
```

5.5 Part 1 — Build inside Docker

The entire build happens in the container. The host needs nothing but Docker — no Java, no Maven.

Create `~/labs-workspace/CalculatorMavenApp/Dockerfile.app1`:

```
FROM maven:3.9-eclipse-temurin-17

WORKDIR /app

# Create a non-root user now; we switch to it after the build
RUN useradd --create-home --shell /bin/bash appuser

# Dependencies first - this layer stays cached unless pom.xml changes
COPY pom.xml .
RUN mvn -B dependency:go-offline

# Source changes invalidate only from here downwards
COPY src ./src
RUN mvn -B clean package

# Hand build output to the unprivileged user, then drop privileges
RUN chown -R appuser:appuser /app
USER appuser

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "/app/target/calculator.jar"]
```

Line by line

`FROM maven:3.9-eclipse-temurin-17` — an official image containing Maven 3.9 **and** Eclipse Temurin JDK 17. Temurin is the Eclipse Foundation's OpenJDK build, the de-facto standard free distribution. This single tag satisfies the lab's "Maven image with Java 17" requirement.

`WORKDIR /app` — sets the working directory for every following instruction, creating it if needed. Always prefer it to `RUN cd /app`, which has no lasting effect: each `RUN` is a separate shell.

`RUN useradd --create-home --shell /bin/bash appuser` — creates the non-root user. `--create-home` gives it `/home/appuser`, which Maven needs for its `~/.m2` cache.

`COPY pom.xml .` **then** `RUN mvn -B dependency:go-offline` — this ordering is the single most valuable Docker technique in the file. Docker caches layers and invalidates **every layer after the first changed one**. Dependencies change rarely; source changes constantly. Copying `pom.xml` alone and downloading dependencies first means editing a `.java` file reuses the cached dependency layer, turning a 3-minute rebuild into 15 seconds. `-B` is batch mode — no ANSI progress spam, standard for CI logs.

`COPY src ./src` **then** `RUN mvn -B clean package` — now the volatile part. Any source edit invalidates from here down only.

`RUN chown -R appuser:appuser /app` then `USER appuser` — ownership transfer, then privilege drop.

> **Why `USER` comes *after* the build.** Maven must write to `/app/target` and `~/.m2`. Put `USER appuser` before `RUN mvn package` and the build fails with permission errors. Put it after, and everything from that point — including the running container — executes unprivileged. This placement question is the heart of the exercise, and its answer differs in Part 2.

`EXPOSE 8080` — metadata only. The app has no listener; this documents intent and satisfies the lab requirement. It opens nothing.

`ENTRYPOINT [...]` — exec form, so `java` becomes PID 1 and receives signals correctly.

Build and measure

```
cd ~/labs-workspace/CalculatorMavenApp && docker build -f Dockerfile.app1 -t app1 .
```

```
docker images app1
```

`-t app1` tags the image. Expect roughly **500–800 MB**. Record the exact figure — the bonus needs it.

Run and verify

```
docker run -it --name container1 app1
```

- `-i` / `--interactive` keeps **stdin open**. Without it the Scanner hits EOF instantly and the app crashes with `NoSuchElementException`.
- `-t` / `--tty` allocates a pseudo-terminal so prompts render properly.
- `--name container1` names the container as the lab requires.

Type `10`, Enter, `5`, Enter. Confirm `Sum: 15.0` and `Multiply: 50.0`.

Try the failure deliberately — understanding it is worth more than avoiding it:

```
docker run --rm app1
```

No `-i` → `NoSuchElementException: No line found`. That is the EOF behaviour, demonstrated.

Confirm the non-root requirement

```
docker run --rm --entrypoint whoami app1
```

Must print `appuser`. `--entrypoint` overrides the image's entrypoint for a one-off command — a genuinely useful debugging technique. `--rm` auto-deletes the container on exit.

Clean up

```
docker rm container1 && docker rmi app1
```

`rm` removes the container, `rmi` removes the image. The container must go first — Docker refuses to delete an image still referenced by a container.

5.6 Part 2 — Build locally

Compile on the host, ship only the finished artifact.

Build the jar on the host

```
cd ~/labs-workspace/CalculatorMavenApp && mvn clean package && ls -lh target/calculator.jar
```

Write `Dockerfile.app2`

```
FROM eclipse-temurin:17-jre-alpine

WORKDIR /app

# Alpine uses BusyBox adduser, not useradd
RUN adduser -D -h /home/appuser appuser

COPY --chown=appuser:appuser target/calculator.jar /app/calculator.jar

USER appuser

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "/app/calculator.jar"]
```

What changed, and why

`FROM eclipse-temurin:17-jre-alpine` — two savings at once:

- `-jre` not `-jdk`: a runtime, no compiler. We are not compiling here.
- `-alpine`: based on Alpine Linux (~5 MB) instead of Ubuntu (~75 MB). Alpine uses `musl` libc and BusyBox utilities rather than `glibc` and GNU coreutils.

> **Alpine caveat worth knowing.** `musl` is not `glibc`. Most Java workloads are unaffected, but applications with native dependencies can behave differently or hit subtle performance issues. Alpine is a sensible default for simple services; verify before using it for something heavy.

`RUN adduser -D -h /home/appuser appuser` — Alpine's BusyBox provides `adduser`, **not** `useradd`. `-D` means no password. Using `useradd` here fails with "command not found". If you choose the Ubuntu-based `eclipse-temurin:17-jre` instead, use `useradd --create-home appuser`. **The base image dictates the available tooling** — a recurring source of confusion.

`COPY --chown=appuser:appuser ...` — sets ownership *during* the copy, avoiding a separate `RUN chown` layer. Fewer layers, smaller image.

`USER appuser` **before** `ENTRYPOINT` — here it can come earlier than in Part 1, because nothing privileged happens afterwards: there is no build, and `COPY --chown` already handled ownership.

Build, measure, verify, clean up

```
cd ~/labs-workspace/CalculatorMavenApp && docker build -f Dockerfile.app2 -t app2 .
```

```
docker images app2
```

Expect roughly **200–300 MB**.

```
docker run -it --name container2 app2
```

```
docker rm container2 && docker rmi app2
```

5.7 Bonus — compare and explain

Capture both sizes **before** deleting the images:

```
docker images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}' | grep -E 'app1|app2|REPOSITORY'
```

Image	Base	Contains	Size
`app1`	`maven:3.9-eclipse-temurin-17`	JDK + Maven + dependency cache + source + jar	~500–800 MB
`app2`	`eclipse-temurin:17-jre-alpine`	JRE + jar	~200–300 MB

The explanation

Build-time dependencies are not run-time dependencies.

Compiling requires a JDK (compiler, debugger, full toolchain), Maven itself, every downloaded dependency, and the complete source tree. *Running* requires a JVM and one 3.7 KB jar. `app1` ships the entire workshop alongside the product; `app2` ships only the product.

Size is the visible symptom. Three consequences matter more:

1. **Attack surface.** Every binary in an image is a potential exploit target. Shipping a compiler, a package manager and a shell hands an attacker who achieves code execution a complete toolkit. Minimal images are a security control, not just an optimization.
2. **Deployment speed.** Image size is pull time — on every node, on every rollout, every time you scale. A 600 MB image across 50 pods is 30 GB of network transfer.
3. **Cost.** Registry storage and egress bandwidth scale with image size, across every build you ever push.

But Part 2 has a real drawback

`app2` is smaller, yet its build is **not reproducible**: it depends on the host having the correct JDK and Maven, and on whatever happened to be in `target/` at build time. Build on a machine with Java 11 and you get a subtly different artifact. Part 1 is fully self-contained — anyone with Docker gets an identical result.

So neither approach is simply "better": **Part 1 buys reproducibility, Part 2 buys size.** Recognising that tension is the point of the lab.

5.8 The real answer — multi-stage builds

Multi-stage builds give you **both**: Part 1's reproducibility at Part 2's size.

Create `Dockerfile.multistage`:

```
# ---- build stage ----
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -B dependency:go-offline
COPY src ./src
RUN mvn -B clean package

# ---- runtime stage ----
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
RUN adduser -D -h /home/appuser appuser
COPY --from=build --chown=appuser:appuser /app/target/calculator.jar /app/calculator.jar
USER appuser
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/calculator.jar"]
```

How it works. Two `FROM` lines mean two stages. The first is named `build` via `AS build`. Only the **final** stage becomes the image. `COPY --from=build` reaches back into the first stage and extracts one file — the jar. Everything else in that stage (JDK, Maven, dependency cache, source) is discarded entirely.

```
cd ~/labs-workspace/CalculatorMavenApp && docker build -f Dockerfile.multistage -t app3 . && docker images app3
```

`app3` should land within a few megabytes of `app2`, while requiring nothing on the host but Docker. **This is the professional answer to the bonus question**, and demonstrating it with a working image is worth considerably more than a paragraph describing it.

5.9 Lab 3 command summary

Command	Purpose
`docker build -f <file> -t <name> .`	build an image
`docker images`	list images with sizes
`docker run -it --name <n> <img>`	run interactively
`docker run --rm <img>`	run and auto-delete
`docker ps` / `docker ps -a`	running / all containers
`docker rm <container>`	delete a container
`docker rmi <image>`	delete an image
`docker history <image>`	show layers and their sizes
`docker system prune -af`	reclaim all unused space

`docker history app1` is worth running — it shows exactly which instruction contributed which megabytes.

---

6. Comparison and conclusions

6.1 The three labs as one story

	Lab 1	Lab 2	Lab 3
Tool	Maven	Gradle	Docker
Input	`.java` source	`.java` source	`.jar` artifact
Output	`target/calculator.jar`	`build/libs/calculator.jar`	an image
Question answered	how do I build?	is there another way?	how do I ship it?

The progression is deliberate: **source → artifact → image**. Each lab consumes the previous one's output. That chain is the core of every CI/CD pipeline you will build later — Jenkins automates it, Kubernetes runs the result, ArgoCD keeps it in sync.

6.2 Decisions and their reasons

Decision	Reason
JDK 17, not a newer one	`pom.xml` targets 17; mismatched JDKs break the build
SDKMAN for Gradle, apt for Maven	Ubuntu's Gradle is from 2017 and too old; its Maven is fine
Clone outside the repo	the submission holds *your* work, not upstream source
`<scope>test</scope>` / `testImplementation`	keeps the test framework out of the runtime artifact
Non-root container user	limits blast radius if the app is compromised
`USER` after build (Part 1), before entrypoint (Part 2)	the build needs write access; the runtime does not
JRE + Alpine base for Part 2	no compiler needed at runtime; smallest sane base
Exec-form `ENTRYPOINT`	the app becomes PID 1 and receives signals correctly
`COPY pom.xml` before `COPY src`	layer caching — dependencies change rarely, source constantly
Multi-stage as a bonus	reproducibility *and* small size, instead of choosing

6.3 Principles worth carrying forward

1. **Build-time ≠ run-time.** The single most transferable idea in these labs.
2. **Convention beats configuration.** Both build tools assume `src/main/java` so you write almost no config.
3. **Fail fast.** A failing test stops packaging. Broken artifacts never get built.
4. **Least privilege.** Processes run with the minimum rights they need. A one-line change with real security value.
5. **Layer order is design, not detail.** It determines build speed and image size.
6. **A green build proves nothing by itself.** `useJUnitPlatform()` missing means zero tests ran and the build still passed.

7. Troubleshooting

Symptom	Cause	Fix
`mvn -version` shows wrong Java	another JDK on `PATH`	`export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64`
Maven very slow on first build	downloading into `~/.m2`	normal; later builds are cached
`no main manifest attribute`	manifest lacks `Main-Class`	check the jar plugin config in `pom.xml`
Gradle: "could not find method toolchain"	Gradle too old (apt version)	install via SDKMAN
Gradle green but zero tests ran	`useJUnitPlatform()` missing	add it to the `test { }` block
`gradle run` prompts unreadable	rich console overwrites output	add `--console=plain -q`
`Cannot connect to the Docker daemon`	daemon not started (no systemd in WSL)	`sudo service docker start`
Docker "permission denied"	user not in `docker` group	`sudo usermod -aG docker $USER`, then reopen the terminal
`NoSuchElementException: No line found`	container stdin closed	use `docker run -i`
`useradd: command not found`	Alpine uses BusyBox	use `adduser -D` instead
Maven permission errors in Docker build	`USER` set before the build	move `USER` after `RUN mvn package`
`docker rmi` refuses	a container still references the image	`docker rm <container>` first
Builds extremely slow in WSL	project is on `/mnt/c`	move it under `/home/<user>`
`fatal: not in a git directory`	`git config` without `--global` needs a repo	run `git init` first
`bash: NAME: No such file or directory`	`<...>` placeholders left in a command	remove the angle brackets — `<` is shell redirection

8. Glossary

Artifact — the distributable file a build produces; here, `calculator.jar`.

Base image — the image a Dockerfile starts `FROM`.

BOM (Bill of Materials) — a dependency that supplies a consistent set of versions for a family of libraries.

Build context — the directory sent to the Docker daemon on `docker build`; `COPY` can only read inside it.

Bytecode — platform-independent instructions in `.class` files, executed by the JVM.

Classpath — where the JVM looks for classes. Separate compile, test and runtime classpaths are why `<scope>` exists.

Container — a running instance of an image; an isolated process sharing the host kernel.

Coordinates — `groupId:artifactId:version`, the unique address of an artifact.

DAG — directed acyclic graph; Gradle's model of tasks and their dependencies.

Image — an immutable, layered filesystem snapshot that containers are created from.

JAR — Java ARchive; a ZIP of `.class` files plus `META-INF/MANIFEST.MF`.

JDK / JRE / JVM — development kit (build) ⊃ runtime environment (run) ⊃ virtual machine (execute).

Layer — the filesystem diff created by a Dockerfile instruction; cached and shared between images.

Lifecycle — Maven's fixed ordered phases: validate → compile → test → package → verify → install → deploy.

Manifest — `META-INF/MANIFEST.MF` inside a JAR; its `Main-Class` entry makes `java -jar` work.

Multi-stage build — a Dockerfile with several `FROM` stages where only the last becomes the image.

musl / glibc — Alpine's C library versus the mainstream one; the reason Alpine images are small and occasionally incompatible.

PID 1 — the first process in a container; it receives signals such as `SIGTERM` from `docker stop`.

Scope — whether a dependency applies at compile, test or runtime.

SDKMAN — version manager for JVM tools (JDKs, Gradle, Maven).

Toolchain — Gradle's declaration of the required JDK, downloaded automatically if absent.

Wrapper (`mvnw` / `gradlew`) — a committed script pinning an exact build-tool version per project.

---

iVolve DevOps Training · Labs 1–3 · Eslam Nabawy